In [8]:
```python
# 1.Code to group all inaugural speeches by each unique president, filter out common grammatical stopwords and punctu

import re
from collections import defaultdict, Counter
import nltk
from nltk.corpus import inaugural

STOPWORDS = {
    'the', 'of', 'and', 'to', 'in', 'a', 'is', 'that', 'for', 'it', 'as', 'was',
    'with', 'be', 'by', 'on', 'not', 'he', 'i', 'this', 'are', 'which', 'but',
    'have', 'our', 'we', 'my', 'you', 'their', 'all', 'they', 'his', 'from',
    'so', 'or', 'at', 'one', 'had', 'been', 'an', 'no', 'will', 'would', 'should',
    'has', 'can', 'there', 'do', 'if', 'them', 'who', 'us', 'your', 'me', 'upon'
}

def get_top_word_per_president():
    president_word_vault = defaultdict(list)

    for fileid in inaugural.fileids():
        match = re.match(r'\d{4}-([A-Za-z\-]+)\.txt', fileid)
        if match:
            president_name = match.group(1)

            words = [w.lower() for w in inaugural.words(fileid) if w.isalpha()]

            filtered_words = [w for w in words if w not in STOPWORDS]

            president_word_vault[president_name].extend(filtered_words)

    print(f"{'President':<20} | {'Most Used Word':<15} | {'Count':<5}")
    print("-" * 46)

    for president, words in sorted(president_word_vault.items()):
        if words:
            word_counts = Counter(words)
            most_common_word, frequency = word_counts.most_common(1)[0]

            print(f"{president:<20} | {most_common_word:<15} | {frequency:<5}")

if __name__ == "__main__":
    get_top_word_per_president()
```

```
President            | Most Used Word  | Count
---------------------------------------------
Adams                | its             | 35
Biden                | s               | 22
Buchanan             | states          | 22
Bush                 | america         | 38
Carter               | nation          | 13
Cleveland            | people          | 40
Clinton              | new             | 38
Coolidge             | its             | 23
Eisenhower           | world           | 32
Garfield             | government      | 23
Grant                | country         | 16
Harding              | world           | 28
Harrison             | people          | 67
Hayes                | country         | 20
Hoover               | government      | 30
Jackson              | government      | 19
Jefferson            | its             | 21
Johnson              | nation          | 12
Kennedy              | let             | 16
Lincoln              | any             | 29
Madison              | war             | 16
McKinley             | its             | 45
Monroe               | great           | 50
Nixon                | let             | 33
Obama                | must            | 25
Pierce               | its             | 14
Polk                 | government      | 47
Reagan               | government      | 34
Roosevelt            | people          | 33
Taft                 | government      | 26
Taylor               | shall           | 15
Truman               | world           | 23
Trump                | america         | 38
VanBuren             | people          | 20
Washington           | every           | 9
Wilson               | own             | 21
```

In [9]:
```python
# 2.Generate a word frequency visualization of a president's inaugural speeches.

import re
import matplotlib.pyplot as plt
import nltk
from nltk.corpus import inaugural
from collections import Counter
```

```
STOPWORDS = {
    "the", "of", "and", "to", "in", "a", "is", "that", "for", "it", "as", "was",
    "with", "be", "by", "on", "not", "he", "i", "this", "are", "which", "but",
    "have", "our", "we", "my", "you", "their", "all", "they", "his", "from",
    "so", "or", "at", "one", "had", "been", "an", "no", "will", "would",
    "has", "can", "there", "do", "if", "them", "who", "us", "your", "me", "upon"
}

def generate_president_wordcloud(target_president):
    text_data = []

    for fileid in inaugural.fileids():
        match = re.match(r"\d{4}-([A-Za-z\-]+)\.txt", fileid)

        if match and match.group(1).lower() == target_president.lower():
            words = [w.lower() for w in inaugural.words(fileid) if w.isalpha()]
            filtered = [w for w in words if w not in STOPWORDS]
            text_data.extend(filtered)

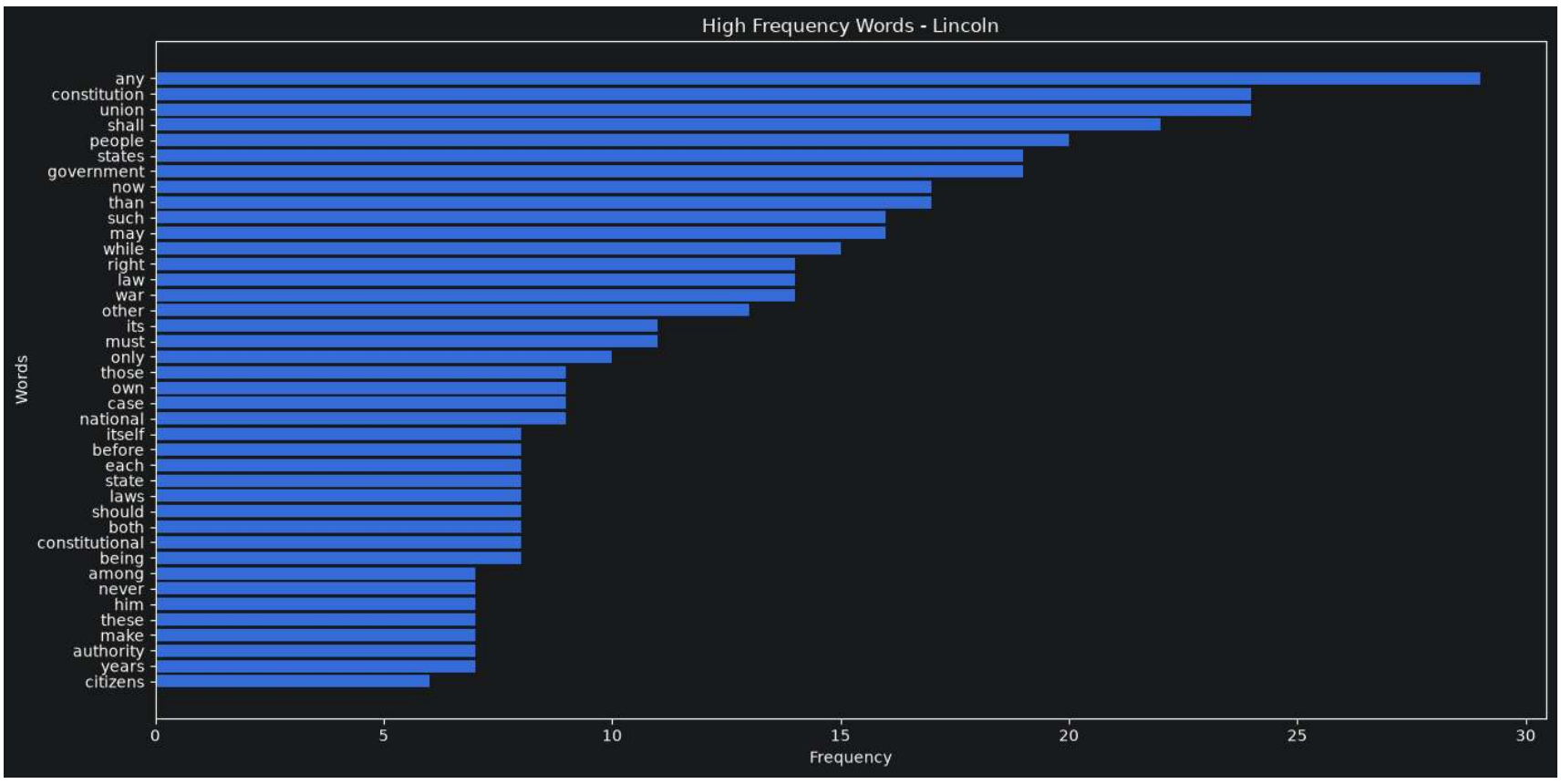
    if not text_data:
        print(f"No speeches found for President: {target_president}")
        return

    word_freq = Counter(text_data)
    common_words = word_freq.most_common(40)

    words = [item[0] for item in common_words]
    frequencies = [item[1] for item in common_words]

    plt.figure(figsize=(14, 7))
    plt.barh(words, frequencies)
    plt.xlabel("Frequency")
    plt.ylabel("Words")
    plt.title(f"High Frequency Words - {target_president.capitalize()}")
    plt.gca().invert_yaxis()
    plt.tight_layout()
    plt.show()

generate_president_wordcloud("Lincoln")
```



In [10]:

```
# 3.Extract COPPA-related news articles from the last 10 years while excluding articles mentioning Obama.

import re

news_articles = [
    "In 2023, the Federal Trade Commission proposed sweeping updates to COPPA to limit targeted advertising to kids u
    "COPPA compliance has become stricter for ed-tech platforms shifting architectures in 2024.",
    "President Obama discussed privacy rights back in his early framework drafts, but current 2026 updates focus on A
    "Meta faced substantial fines regarding COPPA compliance failures over Instagram data processing guidelines."
]

def filter_coppa_news(articles):
    cleaned_feed = []

    for article in articles:
        if not re.search(r"\bobama\b", article, re.IGNORECASE):
            cleaned_feed.append(article)

    return cleaned_feed

filtered_news = filter_coppa_news(news_articles)

print("Filtered COPPA News Feed (No Obama items):")
```

```
for item in filtered_news:
    print(f"- {item}")
```

```
Filtered COPPA News Feed (No Obama items):
- In 2023, the Federal Trade Commission proposed sweeping updates to COPPA to limit targeted advertising to kids unde
r 13.
- COPPA compliance has become stricter for ed-tech platforms shifting architectures in 2024.
- Meta faced substantial fines regarding COPPA compliance failures over Instagram data processing guidelines.
```

In [12]:
```
# 4.Extract unique words containing at least nine characters from Kennedy's inaugural speech.

import nltk
from nltk.corpus import inaugural

def extract_high_sounding_words(fileid, min_length=9):
    raw_words = inaugural.words(fileid)

    unique_words = set(
        w.lower() for w in raw_words if w.isalpha()
    )

    high_sounding = [
        word for word in unique_words if len(word) >= min_length
    ]

    return sorted(high_sounding)

sophisticated_vocab = extract_high_sounding_words("1961-Kennedy.txt")

print(f"Sophisticated vocabulary examples ({len(sophisticated_vocab)} found):")
print(sophisticated_vocab[:15])
```

```
Sophisticated vocabulary examples (67 found):
['accidental', 'administration', 'adversary', 'aggression', 'americans', 'beachhead', 'beginning', 'belaboring', 'cel
ebration', 'challenge', 'committed', 'communists', 'conscience', 'cooperation', 'cooperative']
```

In [18]:
```
# 5.Identify the part-of-speech tag of Pennsylvania using NLTK POS tagging.

import nltk
def tag_pennsylvania_context(sentence_text):
    tokens = nltk.word_tokenize(sentence_text)
    pos_tags = nltk.pos_tag(tokens)

    for word, tag in pos_tags:
        if word.lower() == "pennsylvania":
            print(f"Word: {word} | POS Tag: {tag}")

            if tag == "NNP":
                print("NNP: Proper Noun, Singular")

sample_text = "The historic address was delivered to the delegation in Pennsylvania yesterday."

tag_pennsylvania_context(sample_text)
```

```
Word: Pennsylvania | POS Tag: NNP
NNP: Proper Noun, Singular
```